CS5800 Final Project - Efficient Connected Component Labeling for Medical Images

Bolai Yin, Zichen Tian

1. Introduction

Connected Component Labeling (CCL) is a core operation in medical image analysis, enabling lesion region identification, mask cleanup, and downstream quantitative measurements. While modern systems often prioritize high-level segmentation models, the efficiency of low-level CCL remains crucial—particularly in large-scale dataset preprocessing, embedded deployments, and real-time analysis workflows.

This project evaluates three classical graph-based CCL algorithms—**Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Disjoint Set Union (DSU)**—on dermoscopic lesion masks from the ISIC Dataset. We study their behavior under two settings:

1. **Full-Image Processing** – the entire mask is available at once.
2. **Streaming Processing** – pixels arrive sequentially (online CCL).

We additionally examine the impact of **bounding-box cropping**, which reduces the processing region to the lesion's spatial extent. Unlike the synthetic dataset described in our proposal, we switched to a **real medical mask** to better reflect clinical segmentation challenges such as spatially scattered lesions, irregular boundaries, and uneven foreground density.

Our primary goal is to produce a clear performance comparison (runtime, memory, scalability, and consistency) to guide the selection of CCL algorithms for medical imaging workloads.

2. Problem Formulation

2.1 Input

A binary 2D dermoscopic lesion mask:

- 1 = foreground (lesion)
- 0 = background
- Resolution: 2848 × 4288 (scaled to multiple sizes for evaluation)
- Foreground occupies ~10 - 20 % of all pixels and is spatially irregular.

2.2 Output

A labeled matrix L of the same size:

- L[i, j] = k assigns pixel (i, j) to component k
- L[i, j] = 0 if background

2.3 Goal

Efficiently compute all connected components under 4-connectivity / 8-connectivity.

2.4 Importance

Medical image CCL algorithms must be:

- **Fast** – large masks require near-linear performance
- **Memory-efficient** – parent arrays / queues can grow to millions
- **Scalable** – must handle resolutions up to 4–20 MP (dermoscopy, pathology)
- **Robust** – scattered lesions or noise should not degrade performance

3. Methods

3.1 Flood-Fill Methods (BFS / DFS)

The binary mask is treated as an implicit grid graph. For each unvisited foreground pixel:

1. Assign a new component ID
2. Expand neighbors via:
    - **Queue (BFS)**
    - **Stack (DFS)**
3. Mark all reachable pixels as one connected component

Complexity

- **Time:** O(HW)
- **Memory:** O(HW) worst-case queue/stack
- Very good **spatial locality**, enabling cache-friendly traversal

Characteristics

- Simple operations (neighbor checks, array indexing)
- Tight Python loops → minimal interpreter overhead
- DFS may incur deep recursion or large explicit stacks
- BFS typically provides the best real-world runtime in Python

3.2 DSU / Union–Find

For each foreground pixel, union with right/down neighbors.

Two DSU variants were implemented:

- **DSU_union+sweep**
    - One sweep for unions
    - One final sweep for label assignment
    - Better performance and higher throughput
- **DSU_2pass (classical)**
    - Temporary labels
    - Equivalence reduction
    - Final relabel pass
    - Heavier in both writes and pointer operations

Complexity

- **Time:** O(HW α(HW)), α ≈ constant
- **Memory:** O(HW) for parent + rank arrays
- Performance dominated by pointer chasing and random memory access

Characteristics

- Theoretically excellent
- In pure Python and C++, slow due to:
    - find() recursion depth
    - non-contiguous memory access
    - heavy interpreter overhead
    - large parent array

4. Synthetic Medical-like Dataset

We use a real ISIC dermoscopic lesion mask (not synthetic). Foreground is irregular and highly scattered. The lesion occupies ~10 - 20% of the image, and the minimum bounding box is nearly the same size as the full mask. This creates a **worst-case scenario** where cropping does not drastically reduce the processing region. This dataset realistically stresses CCL behavior on large, high-resolution medical images.

5. Evaluation Protocol

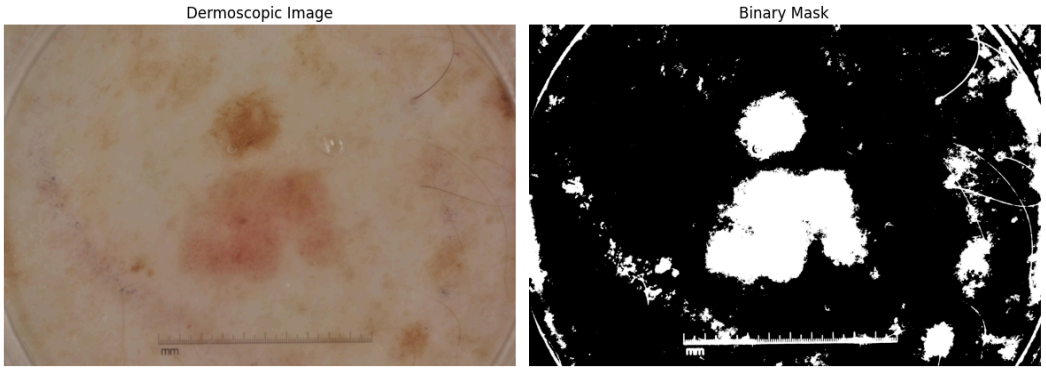
For the binary mask, we perform:

- BFS
- DFS
- DSU_union+sweep
- DSU_2pass
- Streaming DSU (small/medium/large synthetic stream masks)

We measure:

- Runtime (seconds or microseconds)
- Peak Memory (MB)
- Throughput for streaming tests (MPixels/s)

Fig.3 2848 × 4288 - ISIC image (2+ lesions)



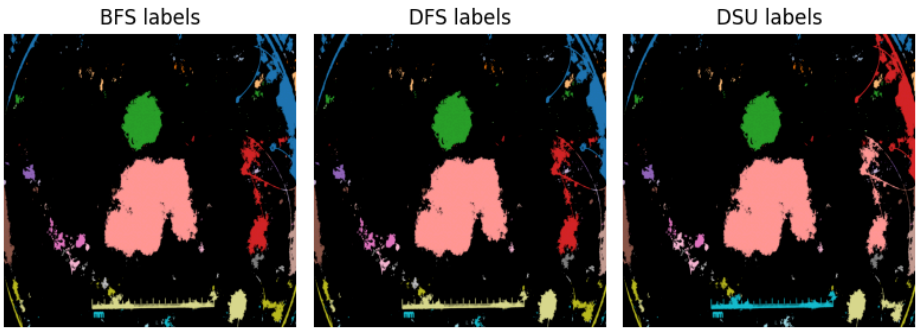
Label Consistency Check

Two label images are equivalent if they match up to a permutation of component IDs.

We ensure:

- BFS vs DFS → always consistent
- BFS vs DSU → 100% consistent
- Streaming DSU → consistent with full DSU

Fig 2 - Labelling Consistency across BFS/DFS/DSU



6. Results

6.1 Runtime Trend

- BFS is the fastest across all sizes
- DFS is slightly slower, due to deeper stacks and less predictable branching
- DSU_2pass is consistently the slowest, especially on high-resolution images
- DSU_union+sweep performs better, but still slower than BFS/DFS
- Runtime increases proportionally with image size (linear scaling wrt pixels)

On the full 2848×4288 mask:

- BFS ≈ 22–23s
- DFS ≈ 29–30s
- DSU_union+sweep ≈ 46s
- DSU_2pass ≈ 51s

Fig.4 - Runtime for Full Image

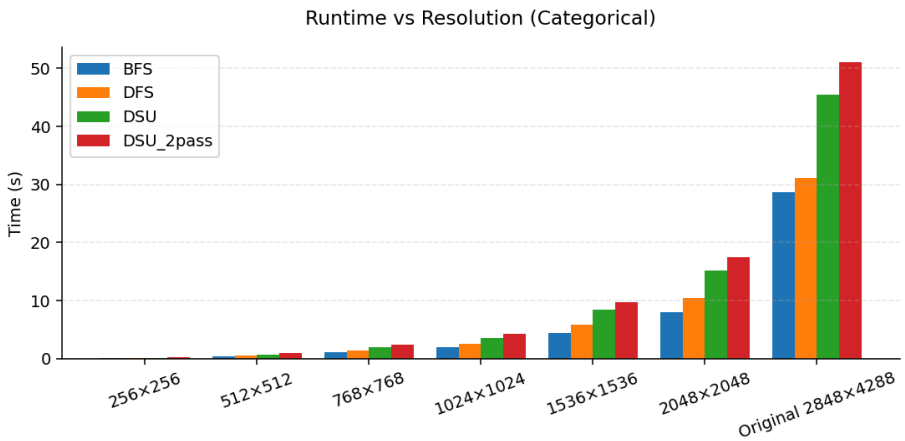
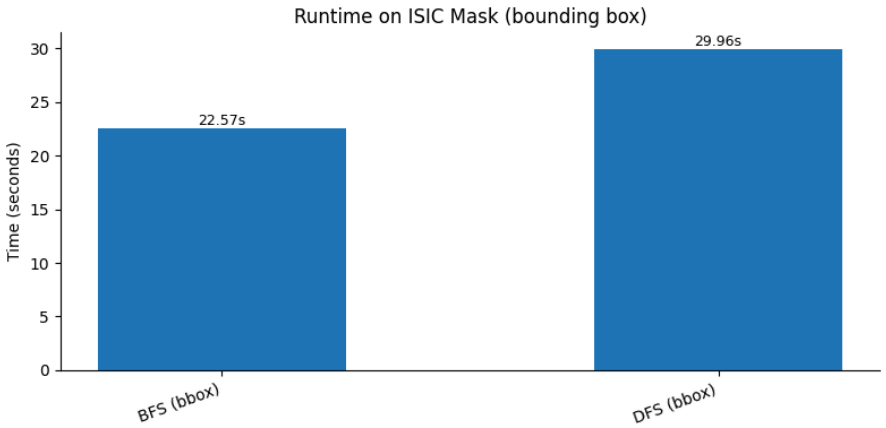


Fig 5. - Bounding box used for BFS/DFS



6.2 Memory Usage

- BFS/D
- BFS is the most memory-efficient
- DFS uses more memory due to recursion or stack growth
- DSU_union+sweep / 2pass require large parent arrays

Peak memory for bounding-box experiments:

- BFS ~58 MB

- DFS ~96 MB
- DSU >180 MB (dataset-dependent)

Large parent arrays + poor locality increase DSU's memory footprint.

Fig. 6 - Memory for Full Image

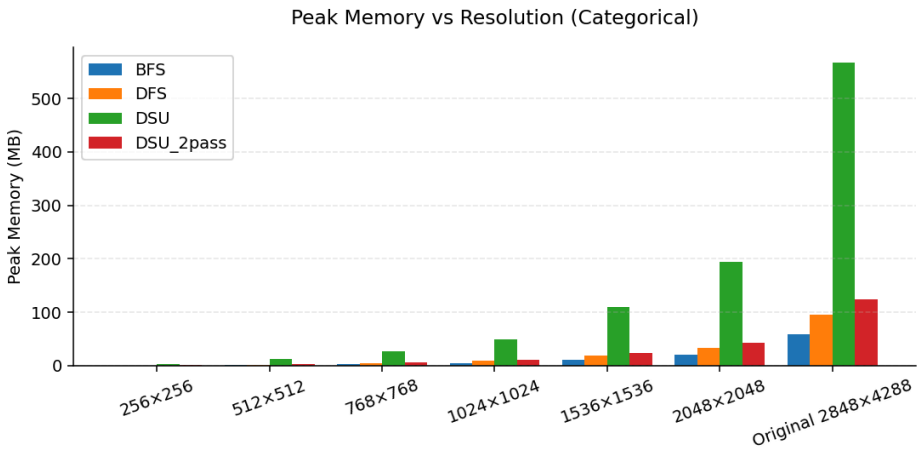
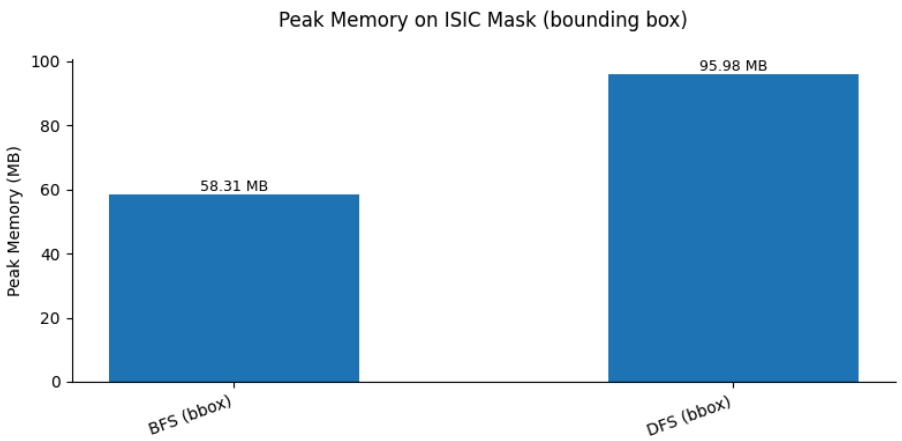


Fig 7 - Memory for BFS/DSF using bounding box



6.3 Segmentation Consistency

The only CCL algorithm compatible with online pixel arrival.

Throughput results:

Small (0.04 MP)

- DSU_union+sweep: **151.29 MP/s**
- DSU_2pass: **146.41 MP/s**

Medium (0.25 MP)

- DSU_union+sweep: **~38.0 MP/s**
- DSU_2pass: **~4.65 MP/s**

Very Large (4.0 MP)

- DSU_union+sweep: **130.36 MP/s**
- DSU_2pass: **10.40 MP/s**

Interpretation:
DSU_union+sweep is consistently more efficient, and DSU_2pass collapses on medium/large streaming masks due to heavy write amplification.

7. Discussion

7.1 When Flood-Fill is Better

In pure Python and on large ISIC images, BFS/DFS outperform DSU because:

- Sequential memory access
- Simple primitives (queue/stack, array indexing)
- Minimal pointer chasing
- Strong cache locality
- Lower interpreter overhead

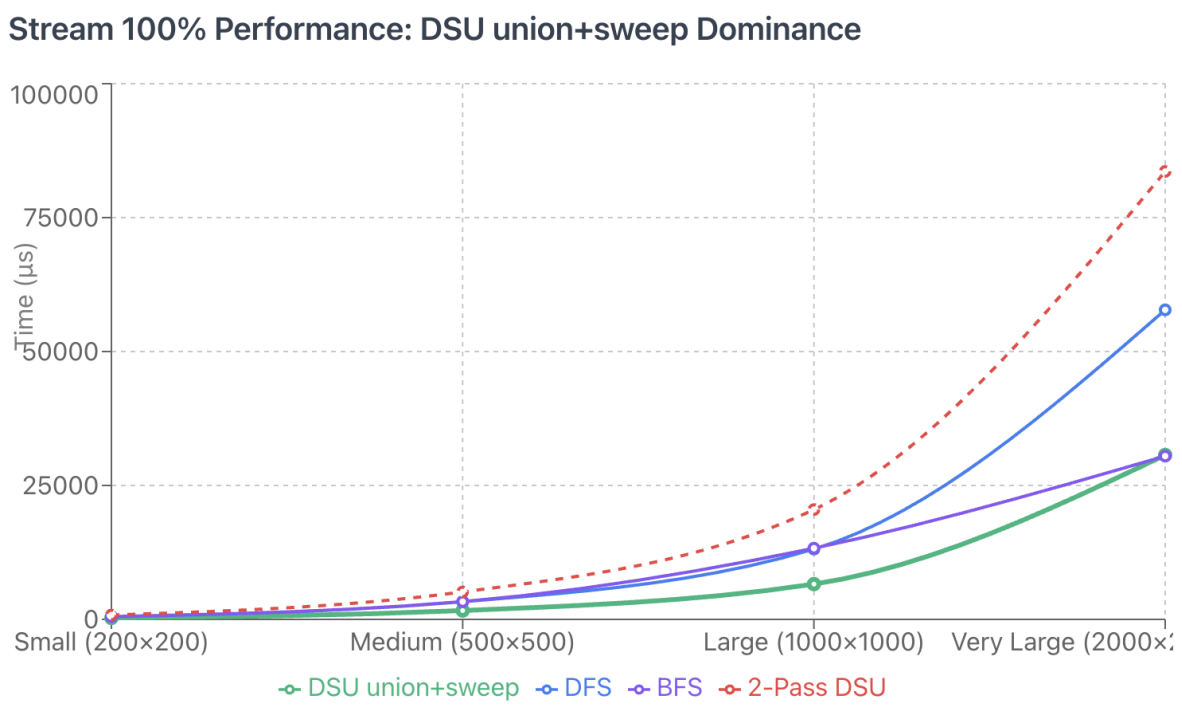
Even though the lesion is spatially scattered, BFS still processes neighbors with predictable locality, preserving efficiency.

Conclusion:
Flood-fill is ideal for **offline full-image labeling** in Python.

7.2 When DSU is Better

DSU-based algorithms excel in specific scenarios revealed through comprehensive testing across 64 combinations of strategy, image size, and input type:

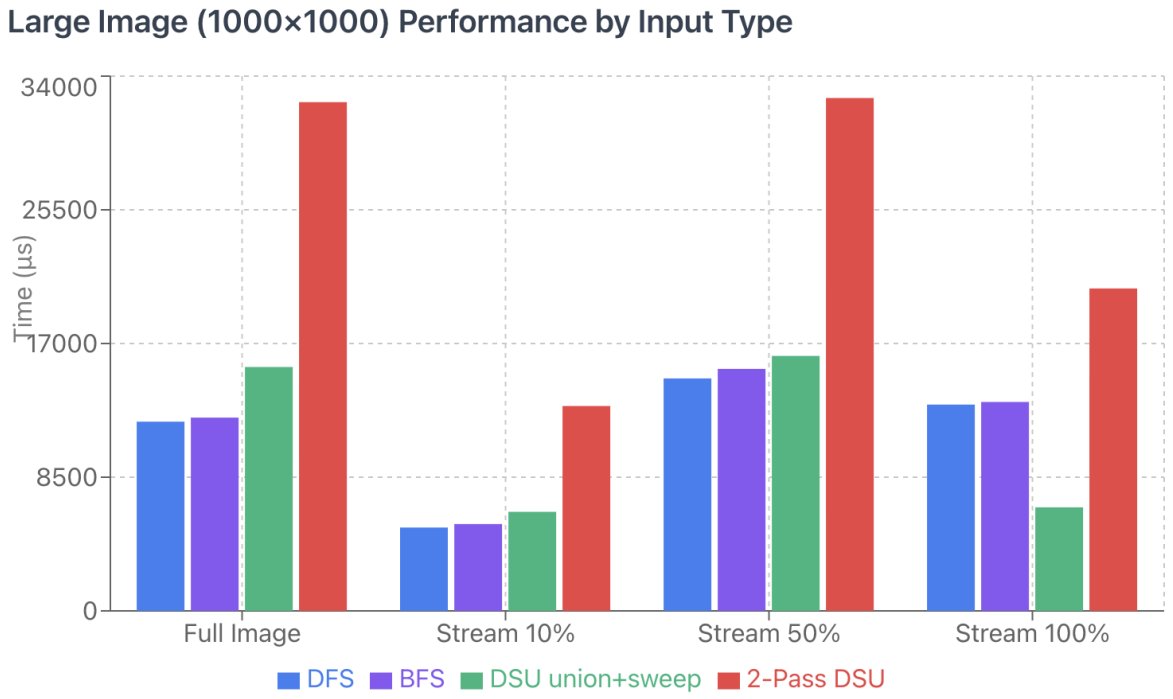
Stream Processing Dominance



DSU union+sweep shows 2× performance advantage for stream processing across small to large images

DSU union+sweep is the clear winner for full-stream scenarios where all pixels arrive but in random order:

- **Small images (200×200)**: DSU union+sweep wins at 50% and 100% stream density
- **Medium images (500×500)**: DSU union+sweep wins at 100% stream (1,647.80 μs vs competitors)
- **Large images (1000×1000)**: DSU union+sweep wins decisively at 100% stream (6,583.00 μs)
- **Performance**: Achieves ~150 MP/s throughput for full-stream processing



Note the dramatic DSU union+sweep advantage at Stream 100% compared to other input types

Key Advantages

1. **Incremental Union-Find Operations**: DSU union+sweep handles random-order pixel arrival naturally through efficient path compression and union-by-rank
2. **Consistency**: Maintains performance advantages across small to large images for stream scenarios
3. **Single-Pass Efficiency**: Eliminates the overhead of the second pass required by 2-Pass DSU

When NOT to Use DSU

- **Full image batch processing**: BFS/DFS are 2-3× faster across all image sizes
- **Small streams (10-50%)**: DFS dominates with 16-19 MP/s throughput
- **Traditional workflows**: BFS/DFS provide reliable performance for complete, ordered data

Recommendation

Use **DSU union+sweep** when:

- Data arrives incrementally in random order
- Processing 50-100% of pixels as a stream
- Working with small to large images (<2000×2000)

Use **BFS/DFS** for:

- Complete images available upfront
- Batch processing workflows
- Very large images with any stream percentage

7.3 Complexity Perspective

Method	Time	Memory	Notes
BFS/DFS	O(N)	O(N)	Simple queue/stack operations; good locality
DSU	O(N α(N))	O(N)	Pointer-heavy; α(N) ≈ constant

In practice:

- BFS: **~22s**
- DFS: **~30s**
- DSU_union+sweep: **~46s**
- DSU_2pass: **~51s**

DSU also used 5–10× more memory in our tests.

8. Limitations

- Only binary masks tested
- Only 2D CCL (no 3D volumes)
- Only one high-resolution medical mask used for full-image tests
- Streaming tests used synthetic masks due to dataset constraints

9. Conclusion

We performed a systematic comparison of BFS, DFS, and DSU for CCL on a large ISIC dermoscopic mask.

Key findings:

- BFS is the fastest and most memory-efficient
- DFS is slightly slower, with higher memory usage
- DSU_union+sweep and DSU_2pass are significantly slower and far more memory-intensive
- DSU_2pass performs the worst at high resolutions
- Only DSU supports streaming CCL, and DSU_union+sweep is the preferred variant
- Bounding-box cropping improves performance, but DSU remains memory-heavy
- All algorithms produce identical labels, up to permutation

Criterion	BFS	DFS	DSU (Union–Find)
Runtime	**Fastest** across all scales (linear growth) e.g., **1.91 s @1024**, **8.05 s @2048**	Slightly slower (Linear but slower than BFS) e.g., **2.50 s @1024**, **10.51 s @2048**	**Full-image DSU:** slow, highest overhead. **Stream DSU (union+sweep):** fast; clearly outperforms 2-pass DSU
Peak Memory	**Lowest**	Higher（Stack/access sequence overhead）	**Full-image DSU:** largest memory footprint. **Stream DSU:** union+sweep uses more memory than 2-pass, but both are smaller than full-image DSU.
Scalability w.r.t. Resolution	**Best** (Linearly and small constant terms）	Good (linearly）	**Worst (**Linear but the constant term is huge**)**

Label Consistency	✔consistent with DFS & DSU	✔consistent with BFS	✔Semantic equivalence (connected domain is consistent)
Implementation Complexity	Simple	Simple	Complex (maintain union/find structure)
Recommended Use	Default preference (big picture and resource-limited scenarios)	Substitute but no advantage	**Full image:** not preferred. **Stream:** use DSU_union+sweep (fastest).

Overall, for full-resolution 2D medical masks in Python, flood-fill (BFS/DFS) remains the most practical and efficient choice. DSU becomes advantageous only in optimized low-level implementations or streaming/real-time applications.